

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 4 : Session – 11 : SLO – 2 | B-Trees — Deletion Algorithm

■ I. Deleting a Leaf Node Value

- Deletion is also done from the LEAF NODES. Two cases: a LEAF NODE value has to be deleted,
- or an INTERNAL NODE value has to be deleted.
- Step 1: Locate the LEAF NODE which has to be deleted.
- Step 2: If the leaf node contains MORE THAN the minimum number of key values (more than $m/2$ elements),
 - then simply delete the value.
- Step 3: Else fill the node by taking an element either from the LEFT sibling or from the RIGHT sibling.
 - a) If the left sibling has more than the MINIMUM NUMBER of key values, push its largest key into its PARENT NODE and pull down the INTERVENING ELEMENT from the parent node to the leaf node.
 - b) Else, if the right sibling has more than the minimum number of key values, push its SMALLEST KEY into its parent node and pull down the intervening element to the LEAF NODE where the key is deleted.
 - c) Else, MERGE a new leaf node by combining the TWO SIBLINGS (leaf + sibling) and the intervening element of the parent node. If pulling the intervening element leaves the parent with less than the minimum number of keys, then PROPAGATE (repeat) the process upwards, thereby REDUCING the height of the B tree.

■ II. Deleting an Internal Node Value

- To delete an internal node, promote the INORDER PREDECESSOR OR INORDER SUCCESSOR of the key to be
- deleted to occupy the position of the deleted key. This predecessor or successor will always be in a
- LEAF NODE. So the processing is done as if a value from the leaf node has been deleted.